CS1200: Intro. to Algorithms and their Limitations Profs. Anshu and Nesson

Problem Set 5

Harvard SEAS - Fall 2026 *Due: Mon Oct. 26, 2026 (11:59pm)*

Please see the syllabus for the full collaboration and generative AI policy, as well as information on grading, late days, and revisions.

All sources of ideas, including (but not restricted to) any collaborators, AI tools, people outside of the course, websites, ARC tutors, and textbooks other than Hesterberg–Vadhan must be listed on your submitted homework along with a brief description of how they influenced your work. You need not cite core course resources, which are lectures, the Hesterberg–Vadhan textbook, sections, SREs, problem sets and solutions sets from earlier in the semester. If you use any concepts, terminology, or problem-solving approaches not covered in the course material by that point in the semester, you must describe the source of that idea. If you credit an AI tool for a particular idea, then you should also provide a primary source that corroborates it. Github Copilot and similar tools should be turned off when working on programming assignments. If you did not have any collaborators or external resources, please write 'none.'

Your name:
Collaborators and External Resources:
No. of late days used on previous psets:
No. of late days used after including this pset:

1. (Shortest Path without predecessor array)

   Home Depot is providing a navigation service for delivery drivers who all leave from the same depot s. The road network is a directed, unweighted graph $G = (V, E)$, with n vertices and m edges, given by outgoing adjacency lists. Every road takes one unit of time to traverse. The graph and depot remain fixed throughout the day. Each query specifies a destination t and asks for a shortest path from s to t, or $\perp$ if none exists.

   Unfortunately, the server that holds the predecessor array has crashed. The backup drive still has the graph, the distance array, and a log of every BFS frontier:

   $$F_i = \{v : \mathrm{dist}_G(s, v) = i\}.$$

   Each frontier is saved as a list of vertex names. Accessing the start of any frontier list takes constant time in the Word-RAM model.

   The engineers deploy the following emergency query algorithm:

   (a) If $\mathsf{dist}[t] = \infty$, return $\perp$.

   (b) Otherwise, set $d = \mathsf{dist}[t]$, start a path list containing t, and set $v = t$.

   (c) For $i = d, d-1, \ldots, 1$, scan the vertices u in F_{i-1} and their outgoing adjacency lists until finding an edge (u, v). Append u to the path list and set $v = u$.

   (d) Reverse the path list and return it.

Every scan starts at the beginning of its list. In rush, the emergency algorithm has ended up answering queries independently: it does not save the predecessors it finds for later queries.

(a) Prove that the emergency algorithm returns a shortest path whenever one exists. Explain why it always finds a suitable vertex in the preceding frontier and why accepting the first match cannot lead to a dead end.

(b) Analyze the time to produce the backup during one BFS, and the worst-case time for one reachable-destination query in terms of n and m. Account for the total work across all frontiers examined during a query. Can a query take $\Omega(n)$ time even when the returned path has only two edges? Justify your answer with a graph and appropriate list orders.

(c) Suppose q queries arrive, with reachable destinations at distances $\ell_1, \ldots, \ell_q$. Compare the total running time, including initial preprocessing, of these approaches:

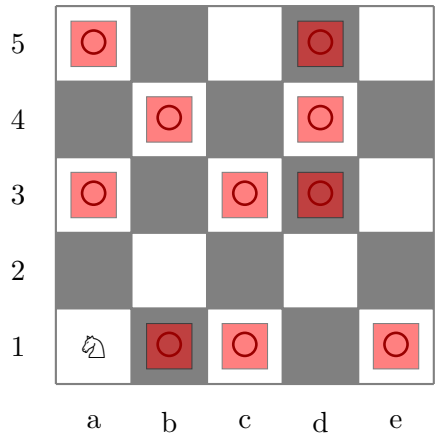
1. Run a fresh BFS for each query.
2. Preprocess once and use the emergency algorithm.
3. Preprocess once and use pred.

Give a family of graphs with $m = \Theta(n)$ and $q = \Theta(n)$ distinct destinations, all at distance two, for which the first two approaches take $\Theta(n^2)$ total time, but the last takes $\Theta(n)$. Specify the list orders used by the emergency algorithm.

2. (Solving Games) Consider playing a solo game on an $n \times n$ chessboard. You have one piece, a chess knight, which starts in the lower-left corner, and your goal is to reach any of the other three corners in as few moves as possible. Like a usual chess knight, in one move, you can move to any position that is two squares away in a horizontal direction and one square away in a vertical direction, or two squares away in a vertical direction and one square away in a horizontal direction. There is a catch, however: some squares have visible landmines, so you cannot move to them (since you do not want set off an explosion).

(a) Give an algorithm that achieves the above goal in time $O(n^2)$ by reduction to either the ShortestWalks problem or the SingleSourceShortestPaths problem. The algorithm should output $\perp$ if no sequence of moves can take you to any of the other three corners when started in the lower-left corner. Discuss correctness and runtime of the algorithm. *Please keep your response within 1 page.*(Note: if reducing to SingleSourceShortestPaths, we haven't defined abstractly what it means to reduce a computational problem to a data structure problem, but you may construct and use a SingleSourceShortestPaths data structure on an appropriately constructed graph.)

(b) Carry out your algorithm on the 5×5 board shown below, listing both the frontier vertices and the predecessor relationships at each stage of BFS (the red square symbols are landmines).



3. (Reductions Between Variants of Independent Set) Consider the following three variants of the IndependentSet problem:

   - Independent Set–Optimization Search: given a graph G, find the largest independent set in G.
   - Independent Set–Threshold Search: given a graph G and a number $k \in \mathbb{N}$, find an independent set of size at least k in G (if one exists).
   - Independent Set–Threshold Decision: given a graph G and a number $k \in \mathbb{N}$, decide (by outputting YES or NO) whether or not there is an independent set of size at least k in G.

   (a) Suppose that there is an algorithm running in time $T(n,m) \geq n+m$ that solves Independent Set–Optimization Search on graphs with at most n vertices and at most m edges. In 8-10 sentences, describe an algorithm running in time $O(T(n,m))$ that solves Independent Set–Threshold Decision. *You* do not *need to prove correctness nor analyze runtime for the algorithm you provide.*

   (b) Suppose that there is an algorithm running in time $T(n,m) \geq n+m$ that solves Independent Set–Threshold Search on graphs with at most n vertices and at most m edges. In 7-8 sentences, describe an algorithm running in time $O((\log n) \cdot T(n,m))$ that solves Independent Set–Optimization Search.

   (c) Suppose that there is an algorithm running in time $T(n,m) \geq n+m$ that solves Independent Set–Threshold Decision. In 8-10 sentences, describe an algorithm running in time $O(n \cdot T(n,m))$ that solves Independent Set–Threshold Search.

   We remark (but you don't need to submit anything) that the combination of the three previous problem parts means that for every constant $c \in [1,2]$, if there is an algorithm solving any one of the three problems in time $(n+m)^{O(1)} \cdot c^n$, there are algorithms solving the other two problems in $(n+m)^{O(1)} \cdot c^n$ time. The best known algorithm (by Xiao and Nagamochi, 2013) has $c \approx 1.1996$.

4. (Maximal Independent Sets) Let $G = (V, E)$ be a graph. A set $S \subseteq V$ is a *maximal* independent set if we cannot add any vertices to S while it remains an independent set. That is, for every vertex $v \in V \setminus S$, $S \cup \{v\}$ is not an independent set.

Given a graph G, a maximal independent set S can be found in time $O(n+m)$ greedily. Specifically, we can start with an arbitrary node in V and add it to S; iterate through each of that node's neighbors and remove them from V; and repeat with the next remaining element in the vertex set until all nodes are exhausted (you can take this fact as a given). Note that this is in sharp contrast to *maximum-size* independent sets, for which we do not know any subexponential-time algorithms.

We know that if G is 3-colorable, then it has a 3-coloring f in which the set of vertices of color 2 (i.e. $f^{-1}(2)$) is a maximal independent set (you can also take this fact as a given). It is also known that every graph G has at most $3^{n/3}$ maximal independent sets, and there is an algorithm (the Bron–Kerbosch algorithm) that enumerates all of the maximal independent sets in time $O(3^{n/3})$. Use these facts to show that a 3-coloring can be solved in time $O((n+m)\cdot 3^{n/3}) = O(1.45^n)$, improving the runtime of $O(1.89^n)$ from SRE2. *Please keep your response within 10 sentences.*

5. (Reflection) Take one homework problem you have worked on this semester that you struggled to understand and solve, and explain how the struggle itself was valuable. In the context of this question, describe the struggle and how you overcame the struggle. You might also discuss whether struggling built aspects of character in you (e.g. endurance, self-confidence, competence to solve new problems) and how these virtues might benefit you in later ventures.

6. Once you're done with this problem set, please fill out this survey so that we can gather students' thoughts on the problem set, and the class in general. It's not required, but we really appreciate all responses! **This week would be a great time to provide any mid-semester feedback, which could help us improve the course in the second half of the semester.**